

ADT Endsem Revision

Recursion, Asymptotic Analysis, Divide and Conquer, Sorting, Dynamic Programming

Vardhan Kumar Ray

Chennai Mathematical Institute

April 14, 2026 - LH801

Outline

- 1 Basic Computational Thinking
- 2 Recursion and Induction
- 3 Asymptotic Analysis
- 4 Divide and Conquer
- 5 Sorting and Loop Invariants
- 6 Dynamic Programming

Basic Computational Thinking - Overview

Core ideas:

- Working with lists and strings at the level of loops, indices, and simple passes.
- Proving correctness using loop invariants or induction on input size.

Question: Maximum Pair Product

Given an array $A[1 \dots n]$ of integers (possibly negative), find two *distinct* indices $i \neq j$ such that $A[i] \cdot A[j]$ is maximised. Design an $\mathcal{O}(n)$ algorithm.

Example:

$$A = [-5, 3, -2, 7, 1] \implies \max \text{ product} = (-5) \cdot (-2) = 10 \text{ or } 3 \cdot 7 = 21.$$

Answer: indices 2, 4 with product 21.

Solution: Maximum Pair Product

Algorithm: In a single pass over A , maintain:

- $\max_1, \max_2$: the two largest values.
- $\min_1, \min_2$: the two smallest values.

After the pass, return $\max(\max_1 \cdot \max_2, \min_1 \cdot \min_2)$.

Correctness: Any pair (i, j) with $i \neq j$ satisfies exactly one of:

- 1 Both $A[i], A[j] \geq 0$: product $\leq \max_1 \cdot \max_2$.
- 2 Both $A[i], A[j] < 0$: product $= |A[i]| \cdot |A[j]| \leq \min_1 \cdot \min_2$.
- 3 One positive, one negative: product ≤ 0 , which is $\leq$ one of the above.

Time: $\mathcal{O}(n)$. **Space:** $\mathcal{O}(1)$.

Question: Lower Median from Upper Median

Let $A[1 \dots n]$ be an array of n *distinct* integers where n is even. You are given the **upper median** m of A (i.e., the $(\frac{n}{2} + 1)$ -th smallest element). Find the **lower median** (the $\frac{n}{2}$ -th smallest element) in $\mathcal{O}(n)$ time.

Example: $A = [7, 2, 9, 4, 5, 1]$, sorted: $[1, 2, 4, 5, 7, 9]$. Upper median = 5 (rank 4). Lower median = 4 (rank 3).

Solution: Lower Median from Upper Median

Key idea: The lower median is the *largest* element among all elements strictly less than m .

Algorithm:

- 1 In one pass, find all elements $< m$. There are exactly $\frac{n}{2}$ such elements (since all are distinct and m has rank $\frac{n}{2} + 1$).
- 2 Return the *maximum* of these elements.

Pseudocode:

```
candidate  $\leftarrow -\infty$ 
for  $i = 1$  to  $n$  do
    if  $A[i] < m$  and  $A[i] > \text{candidate}$  then
        candidate  $\leftarrow A[i]$ 
return candidate
```

Correctness: Since m has rank $\frac{n}{2} + 1$, exactly $\frac{n}{2}$ elements are smaller. The largest of those has rank $\frac{n}{2}$, which is the lower median.

Time: $\mathcal{O}(n)$. **Space:** $\mathcal{O}(1)$.

Recursion - Overview

A **recursive algorithm** solves a problem by reducing it to one or more *smaller instances* of the same problem, with explicit **base cases** for the smallest inputs.

Why induction?

- Every recursive algorithm implicitly has an inductive correctness argument.
- **Base case** $\longleftrightarrow$ base case of induction.
- **Recursive step** $\longleftrightarrow$ inductive step: assume correctness for smaller inputs, prove for current input.

Two flavours:

- **Weak induction:** assume $P(n - 1)$, prove $P(n)$.
- **Strong induction:** assume $P(k)$ for all $k < n$, prove $P(n)$. Needed when the recursion branches or reduces by more than one step.

Question: Binary Exponentiation (n^e)

Given integers n and $e \geq 0$, compute n^e using the **binary exponentiation** (repeated squaring) method. State the algorithm, analyse its time complexity, and prove correctness by induction.

Example: 2^{10} :

$$2^{10} = (2^5)^2, \quad 2^5 = 2 \cdot (2^2)^2, \quad 2^2 = (2^1)^2, \quad 2^1 = 2 \cdot (2^0)^2 = 2.$$

Question: Binary Exponentiation (n^e)

Idea:

$$n^e = \begin{cases} 1 & \text{if } e = 0, \\ (n^{\lfloor e/2 \rfloor})^2 & \text{if } e \text{ is even,} \\ n \cdot (n^{\lfloor e/2 \rfloor})^2 & \text{if } e \text{ is odd.} \end{cases}$$

Solution: Binary Exponentiation - Algorithm & Complexity

Algorithm $\text{POWER}(n, e)$:

if $e = 0$ **return** 1

$t \leftarrow \text{POWER}(n, \lfloor e/2 \rfloor)$

if e is even **return** $t \cdot t$

else return $n \cdot t \cdot t$

Time complexity:

- One recursive call with $\lfloor e/2 \rfloor$; $\mathcal{O}(1)$ work per call.
- Recurrence: $T(e) = T(\lfloor e/2 \rfloor) + \mathcal{O}(1)$.
- Depth of recursion: $\lfloor \log_2 e \rfloor + 1$.

Total: $\mathcal{O}(\log e)$ **multiplications.**

Solution: Binary Exponentiation - Induction Proof

Claim: $\text{POWER}(n, e) = n^e$ for all $e \geq 0$.

Proof by strong induction on e :

Base case ($e = 0$): Returns $1 = n^0$.

Inductive step: Let $e \geq 1$. Assume $\text{POWER}(n, k) = n^k$ for all $0 \leq k < e$. Since $\lfloor e/2 \rfloor < e$, by the inductive hypothesis:

$$t = \text{POWER}(n, \lfloor e/2 \rfloor) = n^{\lfloor e/2 \rfloor}.$$

- If e is even: $e = 2\lfloor e/2 \rfloor$, so the algorithm returns $t^2 = n^{2\lfloor e/2 \rfloor} = n^e$.
- If e is odd: $e = 2\lfloor e/2 \rfloor + 1$, so it returns $n \cdot t^2 = n^{2\lfloor e/2 \rfloor + 1} = n^e$.

By the principle of strong induction, the claim holds for all $e \geq 0$. □

Question: Count LCS of Two Strings

Given two strings $X[1 \dots m]$ and $Y[1 \dots n]$, write a recursive algorithm to compute the **number of distinct longest common subsequences** of X and Y . Analyse time complexity and prove correctness by induction.

Example: $X = AB$, $Y = BA$. LCS length = 1; LCS's are A and B, so count = 2.

Solution: Count LCS – Recurrence

Let $C(i, j)$ = number of distinct LCS's of $X[1 \dots i]$, $Y[1 \dots j]$.

$$C(i, j) = \begin{cases} 1 & \text{if } i = 0 \text{ or } j = 0 \quad (\text{only the empty subsequence}), \\ C(i-1, j-1) & \text{if } X[i] = Y[j], \end{cases}$$

Full case when $X[i] \neq Y[j]$:

$$C(i, j) = \begin{cases} C(i-1, j) & \text{if } L(i-1, j) > L(i, j-1), \\ C(i, j-1) & \text{if } L(i, j-1) > L(i-1, j), \\ C(i-1, j) + C(i, j-1) - C(i-1, j-1) & \text{if } L(i-1, j) = L(i, j-1). \end{cases}$$

The subtraction avoids double-counting LCS's reachable from both $(i-1, j)$ and $(i, j-1)$.

Solution: Count LCS - Complexity & Correctness

Time complexity (naïve recursion):

- The recursion tree can have exponential nodes (overlapping subproblems).
- Naïve: $\mathcal{O}(2^{m+n})$ in worst case.

Correctness (induction on $i + j$):

Base: $i = 0$ or $j = 0$: only the empty string is a common subsequence; count = 1.

Inductive step: Assume correctness for all (i', j') with $i' + j' < i + j$.

- If $X[i] = Y[j]$: every LCS of $X[1..i]$, $Y[1..j]$ must end with $X[i]$, so $C(i, j) = C(i - 1, j - 1)$, which is correct by IH.
- If $X[i] \neq Y[j]$: the set of LCS's is the union of those from $(i - 1, j)$ and $(i, j - 1)$. By inclusion-exclusion, the formula is correct (IH applies to all three sub-states). □

Asymptotic Notation - Definitions

Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$.

- $f(n) = \mathcal{O}(g(n))$: $\exists c > 0, n_0$ such that $f(n) \leq c g(n)$ for all $n \geq n_0$.
- $f(n) = \Omega(g(n))$: $\exists c > 0, n_0$ such that $f(n) \geq c g(n)$ for all $n \geq n_0$.
- $f(n) = \Theta(g(n))$: $f(n) = \mathcal{O}(g(n))$ **and** $f(n) = \Omega(g(n))$.

Useful properties:

- **Transitivity:** $f = \mathcal{O}(g)$ and $g = \mathcal{O}(h) \implies f = \mathcal{O}(h)$.
- For polynomials: $n^a = \mathcal{O}(n^b)$ iff $a \leq b$.
- $\forall k \geq 1, \log^k n = \mathcal{O}(n^\epsilon)$ for any $\epsilon > 0$.

Solving Recurrences - Three Methods

1. Substitution (Estimate & Verify):

- 1 Guess the solution (often by unrolling).
- 2 Prove by strong induction that $T(n) \leq c f(n)$ for appropriate c, n_0 .

2. Recursion Tree: Draw the tree of recursive calls. Sum the work at each level and across all levels.

3. Master Theorem: For $T(n) = a T(n/b) + \Theta(n^d)$ with $a \geq 1, b > 1, d \geq 0$:

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } a < b^d \quad (\text{Case 1: root-dominated}), \\ \Theta(n^d \log n) & \text{if } a = b^d \quad (\text{Case 2: balanced}), \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3: leaf-dominated}). \end{cases}$$

Solution: Substitution Method

Solve the recurrence

$$T(n) = 3T(n/3) + \sqrt{n}, \quad T(1) = 1,$$

using the substitution method.

Estimate. Unrolling the recurrence gives

$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + \sum_{i=0}^{k-1} 3^i \sqrt{\frac{n}{3^i}}.$$

Set $k = \log_3 n$. Then

$$T(n) = n T(1) + \sqrt{n} \sum_{i=0}^{\log_3 n - 1} 3^{i/2} \quad ; \quad \sum_{i=0}^{\log_3 n - 1} 3^{i/2} = \Theta\left(3^{(\log_3 n)/2}\right) = \Theta(\sqrt{n}).$$

$$T(n) = n + \sqrt{n} \cdot \Theta(\sqrt{n}) = \Theta(n).$$

Solution: Substitution Method – Verification

Claim: For suitable constants $c, d > 0$, $T(n) \leq cn - d\sqrt{n}$.

Proof by strong induction on n (assume n is a power of 3).

Base case: For $n = 1$, choose c, d so that

$$T(1) = 1 \leq c - d.$$

Inductive step: Assume the claim holds for $n/3$:

$$T(n) = 3T(n/3) + \sqrt{n} \leq 3 \left(c \frac{n}{3} - d \sqrt{\frac{n}{3}} \right) + \sqrt{n} = cn - (d\sqrt{3} - 1) \sqrt{n}.$$

To finish, we need

$$d\sqrt{3} - 1 \geq d \iff d(\sqrt{3} - 1) \geq 1 \implies d \geq \frac{1}{\sqrt{3} - 1}.$$

Then $T(n) \leq cn - d\sqrt{n}$.

Hence,

$$\boxed{T(n) = \mathcal{O}(n)}.$$

Question: Recursion Tree

Solve $T(n) = 4 T(n/3) + n^2$ using the recursion tree method.

Tree structure: Each node spawns 4 children; problem size shrinks by factor 3.

- **Level 0:** work = n^2 .
- **Level 1:** 4 nodes, each with work $(n/3)^2$; total = $4 \cdot n^2/9 = \frac{4}{9} n^2$.
- **Level k :** 4^k nodes, each with work $(n/3^k)^2$; total = $(4/9)^k n^2$.
- **Depth:** $\log_3 n$ levels.

Solution: Recursion Tree – Summation

Total work:

$$T(n) = n^2 \sum_{k=0}^{\log_3 n} \left(\frac{4}{9}\right)^k = n^2 \cdot \frac{1 - (4/9)^{\log_3 n + 1}}{1 - 4/9}.$$

Since $4/9 < 1$, the geometric series converges:

$$\sum_{k=0}^{\infty} \left(\frac{4}{9}\right)^k = \frac{1}{1 - 4/9} = \frac{9}{5}.$$

Therefore:

$$T(n) \leq \frac{9}{5} n^2 = \Theta(n^2).$$

$$T(n) = \Theta(n^2).$$

Question: Master Theorem Applications

Apply the Master Theorem to solve:

(a) $T(n) = 4 T(n/2) + 10$

(b) $T(n) = 2 T(n/2) + 6n$

Recall: For $T(n) = a T(n/b) + \Theta(n^d)$:

- Case 1 ($a < b^d$): $T(n) = \Theta(n^d)$.
- Case 2 ($a = b^d$): $T(n) = \Theta(n^d \log n)$.
- Case 3 ($a > b^d$): $T(n) = \Theta(n^{\log_b a})$.

Solution: Master Theorem Applications

(a) $T(n) = 4 T(n/2) + 10$:

Here $a = 4$, $b = 2$, $f(n) = 10 = \Theta(n^0)$ so $d = 0$.

$$\log_b a = \log_2 4 = 2 > 0 = d.$$

Case 3 (leaf-dominated): $T(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$.

(b) $T(n) = 2 T(n/2) + 6n$:

Here $a = 2$, $b = 2$, $f(n) = 6n = \Theta(n^1)$ so $d = 1$.

$$\log_b a = \log_2 2 = 1 = d.$$

Case 2 (balanced): $T(n) = \Theta(n^1 \log n) = \Theta(n \log n)$.

Divide and Conquer - Overview

Divide: Split the problem into smaller subproblems.

Conquer: Solve each subproblem recursively (or directly if small enough).

Combine: Merge sub-solutions into the final answer.

Classic D&C algorithms:

- Merge Sort: $T(n) = 2T(n/2) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n \log n)$.
- Karatsuba: $T(n) = 3T(n/2) + \mathcal{O}(n) \Rightarrow \mathcal{O}(n^{\log_2 3}) \approx \mathcal{O}(n^{1.585})$.
- Strassen: $T(n) = 7T(n/2) + \mathcal{O}(n^2) \Rightarrow \mathcal{O}(n^{\log_2 7}) \approx \mathcal{O}(n^{2.807})$.

Question: Search in Rotated Sorted Array

Given an array $A[1 \dots n]$ of *distinct* integers obtained by rotating a sorted array by k positions (unknown), design an $\mathcal{O}(\log n)$ algorithm to determine whether a given element a is present.

Example: Sorted = $[1, 3, 5, 7, 9, 11]$, rotated by $k = 2$: $A = [7, 9, 11, 1, 3, 5]$.
Search for $a = 3$: answer is YES (index 5).

Solution: Search in Rotated Sorted Array

Key observation: In a rotated sorted array, at least one half of any range $[l, r]$ (split at mid) is *always sorted*.

Algorithm ROTATEDSEARCH(A, l, r, a):

```
if  $l > r$  return FALSE
mid  $\leftarrow \lfloor (l + r) / 2 \rfloor$ 
if  $A[mid] = a$  return TRUE
if  $A[l] \leq A[mid]$     % left half is sorted
    if  $A[l] \leq a < A[mid]$  then search  $[l, mid - 1]$ 
    else search  $[mid + 1, r]$ 
else    % right half is sorted
    if  $A[mid] < a \leq A[r]$  then search  $[mid + 1, r]$ 
    else search  $[l, mid - 1]$ 
```

Correctness: At each step, we correctly determine which sorted half a can belong to, and eliminate the other half.

Recurrence: $T(n) = T(n/2) + \mathcal{O}(1) \implies$ **Time:** $\mathcal{O}(\log n)$.

Question: Closest Element in Sorted Array

Given a *sorted* array $A[1 \dots n]$ and a target value a , find the element in A closest to a (i.e., minimising $|A[i] - a|$). Design an $\mathcal{O}(\log n)$ algorithm.

Example: $A = [1, 4, 7, 10, 15]$, $a = 6$. Closest element is 7 (distance 1).

Solution: Closest Element in Sorted Array

Algorithm:

- 1 Run binary search for a in A .
- 2 If a is found, return a .
- 3 Otherwise, binary search gives us the position p where a would be inserted:
 $A[p - 1] < a < A[p]$.
- 4 Return whichever of $A[p - 1]$, $A[p]$ is closer to a . Handle boundary cases ($p = 1$ or $p = n + 1$).

Correctness: Since A is sorted, all elements before $A[p - 1]$ are farther from a than $A[p - 1]$, and all elements after $A[p]$ are farther than $A[p]$. So the closest is one of these two.

Time: $\mathcal{O}(\log n)$.

Question: Maximum Subarray (D&C)

Given an array $A[1 \dots n]$ of integers (positive and negative), find contiguous subarray $A[i \dots j]$ with maximum sum. Design an $\mathcal{O}(n \log n)$ divide-and-conquer algorithm.

Example: $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$. Max subarray = $[4, -1, 2, 1]$ with sum 6.

Divide-and-conquer idea: The maximum subarray either:

- ① lies entirely in the left half,
- ② lies entirely in the right half, or
- ③ **crosses the midpoint.**

Solution: Maximum Subarray (D&C)

Algorithm MAXSUBARRAY(A, l, r):

if $l = r$ **return** $A[l]$

$m \leftarrow \lfloor (l + r)/2 \rfloor$

$L \leftarrow \text{MAXSUBARRAY}(A, l, m)$

$R \leftarrow \text{MAXSUBARRAY}(A, m + 1, r)$

$C \leftarrow \text{MAXCROSSING}(A, l, m, r)$

return $\max(L, R, C)$

MaxCrossing(A, l, m, r): Find max suffix sum of $A[l..m]$ and max prefix sum of $A[m + 1..r]$. Each takes $\mathcal{O}(n)$.

Recurrence: $T(n) = 2 T(n/2) + \mathcal{O}(n)$. By Master Theorem ($a = 2, b = 2, d = 1$, Case 2):
 $T(n) = \mathcal{O}(n \log n)$.

Note: Kadane's algorithm solves this in $\mathcal{O}(n)$.

Question: Merge k Sorted Lists

Given k sorted lists with a *total* of n elements, merge them into one sorted list. Analyse the time complexity of a D&C approach.

Solution: Merge k Sorted Lists

Approach 1 (Naïve): Merge lists pairwise one by one. Can be $\mathcal{O}(nk)$ in the worst case.

Approach 2 (D&C): Divide the k lists into two groups of $k/2$, recursively merge each group, then merge the two resulting sorted lists.

D&C Algorithm:

if $k = 1$ **return** the single list

Recursively merge lists $1, \dots, \lfloor k/2 \rfloor$ into L_1 .

Recursively merge lists $\lfloor k/2 \rfloor + 1, \dots, k$ into L_2 .

Merge L_1 and L_2 in $\mathcal{O}(n)$ time (standard merge).

Recurrence (on k): At each level we do $\mathcal{O}(n)$ total merge work (every element participates in exactly one merge at each level). Number of levels: $\log_2 k$.

$$T(n, k) = \mathcal{O}(n \log k) \implies \text{Time: } \mathcal{O}(n \log k).$$

Question: Count Inversions (D&C)

Given an array $A[1 \dots n]$, an **inversion** is a pair (i, j) with $i < j$ and $A[i] > A[j]$. Count the total number of inversions in $\mathcal{O}(n \log n)$ time.

Example: $A = [2, 4, 1, 3, 5]$. Inversions: $(2, 1), (4, 1), (4, 3)$. Count = 3.

Idea: Modify merge sort. Inversions are of three types:

- 1 Both in left half (counted recursively).
- 2 Both in right half (counted recursively).
- 3 **Split inversions:** i in left, j in right (counted during merge).

Solution: Count Inversions

During the **merge step**, when we pick an element from the right half $R[j]$ before some elements in the left half $L[i..m]$, all remaining elements in $L[i..m]$ form inversions with $R[j]$.

Counting split inversions: Each time we pick from the right half in the merge, add (remaining elements in left half) to the inversion count.

Recurrence: Same as merge sort: $T(n) = 2T(n/2) + \mathcal{O}(n)$.

Time: $\mathcal{O}(n \log n)$.

Correctness: Every inversion is counted exactly once, in the merge step at the level where the two elements are split across the left and right halves.

Sorting Algorithms - Quick Reference

Algorithm	Best	Worst	Notes
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	Stable, in-place
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	In-place, not stable
Merge Sort	$\mathcal{O}(n \log n)$		Stable, $\mathcal{O}(n)$ space
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	In-place, expected $\mathcal{O}(n \log n)$
Counting Sort	$\mathcal{O}(n + k)$		Keys in $\{0, \dots, k\}$, stable
Radix Sort	$\mathcal{O}(d(n + k))$		d digits, base k

Lower bound: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case (decision-tree argument).

Loop Invariants

A **loop invariant** is a property that holds:

- 1 **Initialization:** True before the first iteration.
- 2 **Maintenance:** If true before an iteration, it remains true after.
- 3 **Termination:** When the loop ends, the invariant gives us the desired result.

Example - Insertion Sort invariant:

“At the start of iteration i , the subarray $A[1 \dots i - 1]$ is a sorted permutation of the elements originally in $A[1 \dots i - 1]$.”

- *Init:* Before $i = 2$, $A[1 \dots 1]$ is trivially sorted.
- *Maint:* Inserting $A[i]$ into its correct position preserves sortedness.
- *Term:* At $i = n + 1$, $A[1 \dots n]$ is sorted.

Non-Comparison Sorts

Counting Sort ($\mathcal{O}(n + k)$):

- Input: $A[1 \dots n]$ with keys in $\{0, 1, \dots, k\}$.
- Count occurrences of each key; compute prefix sums; place elements in output.
- **Stable:** equal keys preserve their relative order.
- Linear when $k = \mathcal{O}(n)$.

Radix Sort ($\mathcal{O}(d(n + k))$):

- Sort by *least significant digit* first, using a *stable* sort (e.g., counting sort).
- Repeat for each of d digits.
- Correctness: stability ensures that ties on digit i are broken correctly by digit $i - 1$.

These sorts break the $\Omega(n \log n)$ barrier because they exploit structure (integer keys), not just comparisons.

Question: k -Sorting an Array

An array A is called **k -sorted** if for all valid i :

$$\frac{A[i] + A[i + 1] + \cdots + A[i + k - 1]}{k} \leq \frac{A[i + 1] + A[i + 2] + \cdots + A[i + k]}{k}.$$

Design an algorithm that k -sorts an array $A[1 \dots n]$ in $\mathcal{O}(n \log(n/k))$ time.

Solution: k -Sorting an Array

Observation: The condition $A[i] \leq A[i + k]$ means: elements at indices $\equiv r \pmod{k}$ (for each r) must be sorted among themselves.

Algorithm:

- ① For each $r = 0, 1, \dots, k - 1$:
 - Extract the subsequence $B_r = A[r + 1], A[r + 1 + k], A[r + 1 + 2k], \dots$
 - Sort B_r (which has $\lceil n/k \rceil$ or $\lfloor n/k \rfloor$ elements).
 - Write the sorted values back to their positions.

Time complexity:

- Each B_r has $\Theta(n/k)$ elements. Sorting it takes $\mathcal{O}((n/k) \log(n/k))$.
- There are k such subsequences.
- Total: $k \cdot \mathcal{O}((n/k) \log(n/k)) \implies$ **Time:** $\mathcal{O}(n \log(n/k))$.

Correctness: After sorting each residue class, $A[i] \leq A[i + k]$ for all i , satisfying the k -sorted property.

Question: Find a Duplicate in $\mathcal{O}(n \log n)$

Given an array $A[1 \dots n]$ of integers, determine if there is a duplicate element (i.e., $A[i] = A[j]$ for some $i \neq j$), using $\mathcal{O}(n \log n)$ time and $\mathcal{O}(1)$ extra space (or $\mathcal{O}(n)$ with sorting).

Sort and scan.

- 1 Sort A in $\mathcal{O}(n \log n)$ time.
- 2 Scan adjacent pairs: if $A[i] = A[i + 1]$ for some i , return `TRUE`.

Time: $\mathcal{O}(n \log n)$.

Dynamic Programming - Overview

Key ingredients:

- 1 **State:** A parameterisation of subproblems.
- 2 **Base cases:** Values for the smallest subproblems.
- 3 **Transition:** How to compute a state from smaller states (the recurrence).
- 4 **Order:** Fill states in an order such that all dependencies are already computed.

Top-down (Memoization): Write the recursion naturally; cache results.

Bottom-up: Fill a table iteratively in increasing order of subproblem size.

When to use DP:

- **Optimal substructure:** optimal solution contains optimal solutions to subproblems.
- **Overlapping subproblems:** the same subproblem is solved multiple times.

Standard DP Examples – Quick Reference

Problem	State	Time
Fibonacci	$F[n]$	$\mathcal{O}(n)$
Rod Cutting	$R[j]$ (length j)	$\mathcal{O}(n^2)$
LIS	$L[i]$ (ending at i)	$\mathcal{O}(n^2)$
LCS	$L[i][j]$	$\mathcal{O}(mn)$
Edit Distance	$D[i][j]$	$\mathcal{O}(mn)$
0-1 Knapsack	$K[i][w]$	$\mathcal{O}(nW)$
MCM	$M[i][j]$	$\mathcal{O}(n^3)$

For every DP problem, clearly define: (1) what the state represents, (2) the recurrence, (3) the base cases, (4) the order of computation, and (5) the answer location in the table.

LIS - Longest Increasing Subsequence

State: $L[i]$ = length of the longest increasing subsequence *ending at* index i .

Transition:

$$L[i] = 1 + \max_{\substack{j < i \\ A[j] < A[i]}} L[j], \quad L[i] = 1 \text{ if no such } j \text{ exists.}$$

Base: $L[i] = 1$ for all i (every element is an IS of length 1).

Answer: $\max_{1 \leq i \leq n} L[i]$.

Time: $\mathcal{O}(n^2)$. (Can be improved to $\mathcal{O}(n \log n)$ with patience sorting.)

Example: $A = [3, 1, 4, 1, 5, 9, 2, 6]$.

$$L = [1, 1, 2, 1, 3, 4, 2, 4]. \quad \text{LIS length} = 4 \text{ (e.g., } 1, 4, 5, 9\text{).}$$

LCS - Longest Common Subsequence

State: $C[i][j]$ = LCS length of $X[1 \dots i]$ and $Y[1 \dots j]$.

Transition:

$$C[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0, \\ C[i-1][j-1] + 1 & X[i] = Y[j], \\ \max(C[i-1][j], C[i][j-1]) & X[i] \neq Y[j]. \end{cases}$$

Base: $C[0][j] = C[i][0] = 0$ for all i, j .

Answer: $C[m][n]$.

Time: $\mathcal{O}(mn)$, **Space:** $\mathcal{O}(mn)$ (can be reduced to $\mathcal{O}(\min(m, n))$).

Reconstruct the LCS: Trace back from $C[m][n]$: go diagonally when $X[i] = Y[j]$, otherwise go in the direction of the larger value.

Edit Distance

State: $D[i][j]$ = minimum edit distance between $X[1 \dots i]$ and $Y[1 \dots j]$.

Transition:

$$D[i][j] = \begin{cases} j & i = 0, \\ i & j = 0, \\ D[i-1][j-1] & X[i] = Y[j], \\ 1 + \min(D[i-1][j], D[i][j-1], D[i-1][j-1]) & X[i] \neq Y[j]. \end{cases}$$

The three options in the min correspond to: *delete* from X , *insert* into X , *replace* in X .

Time: $\mathcal{O}(mn)$.

Rod Cutting

Given a rod of length n and a price table $P[1 \dots n]$ where $P[i]$ is the price of a rod of length i , determine the maximum revenue from cutting the rod.

State: $R[j]$ = maximum revenue obtainable from a rod of length j .

Transition:

$$R[j] = \max_{1 \leq i \leq j} \{P[i] + R[j - i]\}.$$

Base: $R[0] = 0$.

Order: Fill $R[1], R[2], \dots, R[n]$.

Time: $\mathcal{O}(n^2)$.

Naïve recursion: $T(n) = 2^n$ (exponential). Memoization or bottom-up both give $\mathcal{O}(n^2)$.

0-1 Knapsack

Given n items with weights $w_1, \dots, w_n$ and values $v_1, \dots, v_n$, and a capacity W , maximise total value without exceeding W . Each item is used at most once.

State: $K[i][w]$ = max value using items $\{1, \dots, i\}$ with capacity w .

Transition:

$$K[i][w] = \begin{cases} K[i-1][w] & w_i > w \text{ (can't take item } i), \\ \max(K[i-1][w], v_i + K[i-1][w - w_i]) & w_i \leq w. \end{cases}$$

Base: $K[0][w] = 0$ for all w ; $K[i][0] = 0$ for all i .

Answer: $K[n][W]$.

Time: $\mathcal{O}(nW)$ (pseudo-polynomial).

Question: Equal Sum Partition

Given a multiset $S = \{a_1, a_2, \dots, a_n\}$ of positive integers with total sum Σ , determine whether S can be partitioned into two subsets with equal sum. If Σ is odd, the answer is NO. Otherwise, construct a DP solution in $\mathcal{O}(n \cdot \Sigma)$ time.

Example: $S = \{1, 5, 11, 5\}$, $\Sigma = 22$. Target = 11. Partition: $\{1, 5, 5\}$ and $\{11\}$.

Reduction: This is equivalent to: does there exist a subset of S summing to $\Sigma/2$? This is a **Subset Sum** instance.

Solution: Equal Sum Partition – DP

Target: $T = \Sigma/2$ (if Σ is odd, return NO).

State: $\text{dp}[i][s] = \text{TRUE}$ iff some subset of $\{a_1, \dots, a_i\}$ sums to s .

Transition:

$$\text{dp}[i][s] = \text{dp}[i-1][s] \vee (s \geq a_i \wedge \text{dp}[i-1][s-a_i]).$$

Base: $\text{dp}[0][0] = \text{TRUE}$; $\text{dp}[0][s] = \text{FALSE}$ for $s > 0$.

Answer: $\text{dp}[n][T]$.

Time: $\mathcal{O}(n \cdot \Sigma)$

Question: Palindromic Decomposition

Given a string $s[1 \dots n]$, determine whether s can be written as a concatenation of *non-trivial* palindromic substrings (each of length ≥ 2).

Example:

- $s = \text{aabb aa}$: Yes. $\text{aa} + \text{bb} + \text{aa}$ (or aabb aa itself).
- $s = \text{abc}$: No, no palindrome of length ≥ 2 can start with ab .

Solution: Palindromic Decomposition

Step 1: Precompute palindromes.

$\text{isPal}[i][j] = \text{TRUE}$ iff $s[i \dots j]$ is a palindrome. Compute in $\mathcal{O}(n^2)$ using the recurrence:

$$\text{isPal}[i][j] = (s[i] = s[j]) \wedge (j - i \leq 2 \vee \text{isPal}[i+1][j-1]).$$

Step 2: DP for decomposition.

State: $\text{dp}[i] = \text{TRUE}$ iff $s[1 \dots i]$ can be decomposed into non-trivial palindromes.

Transition:

$$\text{dp}[i] = \bigvee_{\substack{0 \leq j \leq i-2 \\ \text{isPal}[j+1][i]}} \text{dp}[j].$$

(We try every palindromic suffix of length ≥ 2 .)

Base: $\text{dp}[0] = \text{TRUE}$ (empty prefix).

Answer: $\text{dp}[n]$.

Time: $\mathcal{O}(n^2)$.

Question: All Possible Sums with Coins

Given n coins with denominations $c_1, c_2, \dots, c_n$ (not necessarily distinct), each coin can be used **at most once**. Find all possible sums that can be formed.

Example: Coins = $\{1, 2, 2, 5\}$. Possible sums: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$.

Solution: All Possible Coin Sums

Let $\Sigma = \sum_{i=1}^n c_i$.

State: $\text{dp}[s] = \text{TRUE}$ iff sum s is achievable using some subset of the coins.

Algorithm (bottom-up, 1D):

$\text{dp}[0] \leftarrow \text{TRUE}; \quad \text{dp}[s] \leftarrow \text{FALSE} \text{ for } s = 1, \dots, \Sigma$

for $i = 1$ **to** n :

for $s = \Sigma$ **down to** c_i : % reverse order to use each coin once

$\text{dp}[s] \leftarrow \text{dp}[s] \vee \text{dp}[s - c_i]$

Answer: $\{s : \text{dp}[s] = \text{TRUE}\}$.

Time: $\mathcal{O}(n \cdot \Sigma)$, **Space:** $\mathcal{O}(\Sigma)$.

Matrix Chain Multiplication

Given matrices $M_1, M_2, \dots, M_n$ with dimensions $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$, find the parenthesisation that minimises the total number of scalar multiplications.

State: $\text{OPT}[i][j] = \text{min cost to multiply } M_i \cdot M_{i+1} \cdots M_j$.

Transition:

$$\text{OPT}[i][j] = \min_{i \leq k < j} \{ \text{OPT}[i][k] + \text{OPT}[k+1][j] + p_{i-1} \cdot p_k \cdot p_j \}.$$

Base: $\text{OPT}[i][i] = 0$ for all i .

Order: Fill by increasing chain length $\ell = j - i + 1$, from $\ell = 2$ to n .

Answer: $\text{OPT}[1][n]$.

Time: $\mathcal{O}(n^3)$, **Space:** $\mathcal{O}(n^2)$.

Bye Bye!

You can find the Revision Handout here:

<https://drive.google.com/file/d/1ScscrKaQPXJ-8CTMuQMxdSHWYpTLk239>

All the best for the exam!